

Foundations of Mixture of experts

[Mixture of experts]

1.0 Content Block

For each input-output pair (x, y) , the weighting function is changed to increase the weight on all experts that performed above average, and decrease the weight on all experts that performed below average. This encourages the weighting function to learn to select only the experts that make the right predictions for each input. The i -th expert is changed to make its prediction closer to y , but the amount of change is proportional to $w(x) N(y | \mu_i, I)$. This has a Bayesian interpretation. Given input x , the prior probability that expert i is the right one is $w(x)_i$, and $N(y | \mu_i, I)$ is the likelihood of evidence y . So, $w(x)_i N(y | \mu_i, I) / \sum_j w(x)_j N(y | \mu_j, I)$ is the posterior probability for expert i , and so the rate of change for the i -th expert is proportional to its posterior probability. In words, the experts that, in hindsight, seemed like the good experts to consult, are asked to learn on the example. The experts that, in hindsight, were not, are left alone. The combined effect is that the experts become specialized: Suppose two experts are both good at predicting a certain kind of input, but one is slightly better, then the weighting function would eventually learn to favor the better one. After that happens, the lesser expert is unable to obtain a high gradient signal, and becomes even worse at predicting such kind of input. Conversely, the lesser expert can become better at predicting other kinds of input, and increasingly pulled away into another region. This has a positive feedback effect, causing each expert to move apart from the rest and take care of a local region alone (thus the name "local experts").

2.0 Content Block

Hierarchical MoE Hierarchical mixtures of experts uses multiple levels of gating in a tree. Each gating is a probability distribution over the next level of gatings, and the experts are on the leaf nodes of the tree. They are similar to decision trees. For example, a 2-level hierarchical MoE would have a first order gating function w_i , and second order gating functions $w_{j|i}$ and experts $f_{j|i}$. The total prediction is then $\sum_i w_i(x) \sum_j w_{j|i}(x) f_{j|i}(x)$.

3.0 Content Block

Deep learning The previous section described MoE as it was used before the era of deep learning. After deep learning, MoE found applications in running the largest models, as a simple way to perform conditional computation: only parts of the model are used, the parts chosen according to what the input is. The earliest paper that applies MoE to deep learning dates back to 2013, which proposed to use a different gating network at each layer in a deep neural network. Specifically, each gating is a linear-ReLU-linear-softmax network, and each expert is a linear-ReLU network. Since the output from the gating is not sparse, all expert outputs are needed, and no conditional computation is performed. The key goal when using MoE in deep learning is to reduce computing cost. Consequently, for each query, only a small subset of the experts should be queried. This makes MoE in deep learning different from classical MoE. In classical MoE, the output for each query is a weighted sum of all experts' outputs. In deep learning MoE, the output for each query can only involve a few experts' outputs. Consequently, the key design choice in MoE becomes routing: given a batch of queries, how to route the queries to the best experts.

4.0 Content Block

$$\nabla_{\mu_i} \ln \pi_{\theta}(y|x) = w(x)_i N(y|\mu_i, I) \sum_j w(x)_j N(y|\mu_j, I) (y - \mu_i) \quad \nabla_{\theta_i} \ln \pi_{\theta}(y|x) = \frac{w(x)_i N(y|\mu_i, I)}{\sum_j w(x)_j N(y|\mu_j, I)} (y - \mu_i)$$

5.0 Content Block

Experts $f_1, \dots, f_n$, each taking the same input x , and producing outputs $f_1(x), \dots, f_n(x)$. A weighting function (also known as a gating function) w , which takes input x and produces a vector of outputs $(w(x)_1, \dots, w(x)_n)$. This may or may not be a probability distribution, but in both cases, its entries are non-negative.

6.0 Content Block

and the gradient for the weighting function is $\nabla_{[k_i, b_i]} \ln \pi_{\theta}(y|x) = [x_1] w(x)_i \sum_j w(x)_j N(y|\mu_j, I) (f_i(x) - f_{\theta}(y|x))$

7.0 Content Block

Sparsely-gated MoE layer The sparsely-gated MoE layer, published by researchers from Google Brain, uses feedforward networks as experts, and linear-softmax gating. Similar to the previously proposed hard MoE, they achieve sparsity by a weighted sum of only the top- k experts, instead of the weighted sum of all of them. Specifically, in a MoE layer, there are feedforward networks $f_1, \dots, f_n$, and a gating network w . The gating network is defined by $w(x) = \text{softmax}(\text{top}_k(Wx + \text{noise}))$, where top_k is a function that keeps the top- k entries of a vector the same, but sets all other entries to $-\infty$. The addition of noise helps with load balancing. The choice of k is a hyperparameter that is chosen according to application. Typical values are $k = 1, 2$. The $k = 1$ version is also called the Switch Transformer. The original Switch Transformer was applied to a T5 language model. As demonstration, they trained a series of models for machine translation with alternating layers of MoE and LSTM, and compared with deep LSTM models. Table 3 shows that the MoE models used less inference time compute, despite having 30x more parameters. This architectural module was published in 2017-01, within a few months of the publication of the Transformer architecture (2017-06-12), and they were combined into a multimodal architecture called MultiModel published 4 days later (2017-06-16).